HOMEWORK SET 4

Name: Rameez Hamza **Roll No:** 2025115140

1. Implement a Stack Using Queue Data Structure

```
#include <stdio.h>

#define MAX 50

int q[MAX], front = 0, rear = 0;

void push(int x) {
    int i, temp[MAX], n = rear - front;

    temp[0] = x;
    for (i = 0; i < n; i++)
        temp[i + 1] = q[front + i];

    for (i = 0; i <= n; i++)
        q[i] = temp[i];

    front = 0;
    rear = n + 1;
}

int pop() {
    if (front == rear)
        return -1;
    return q[front++];
}

int main() {
    push(14);
    push(27);
    push(39);

    printf("Popped: %d\n", pop());
    printf("Popped: %d\n", pop());
    printf("Popped: %d\n", pop());

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Popped: 39
Popped: 27
Popped: 14
```

2. Implement a Queue Using Stack Data Structure

```
#include <stdio.h>

#define MAX 50

int s1[MAX], s2[MAX];
int top1 = -1, top2 = -1;

void enqueue(int x) {
    s1[++top1] = x;
}

int dequeue() {
    if (top1 == -1 && top2 == -1)
        return -1;

    if (top2 == -1)
        while (top1 >= 0)
            s2[++top2] = s1[top1--];

    return s2[top2--];
}

int main() {
    enqueue(11);
    enqueue(22);
    enqueue(33);

    printf("Dequeued: %d\n", dequeue());
    printf("Dequeued: %d\n", dequeue());
    printf("Dequeued: %d\n", dequeue());

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Dequeued: 11
Dequeued: 22
Dequeued: 33
```

3. Browser History Using Stack

```
#include <stdio.h>
#include <string.h>

#define MAX 20

char back[MAX][50], forward[MAX][50];
int bt = -1, ft = -1;
char current[50] = "Home";

void visit(char page[]) {
    strcpy(back[++bt], current);
    strcpy(current, page);
    ft = -1;
}

void backPage() {
    if (bt < 0) return;
    strcpy(forward[++ft], current);
    strcpy(current, back[bt--]);
}

void forwardPage() {
    if (ft < 0) return;
    strcpy(back[++bt], current);
    strcpy(current, forward[ft--]);
}

int main() {
    visit("Google");
    visit("GitHub");
    visit("Wikipedia");

    printf("Current: %s\n", current);

    backPage();
    printf("After Back: %s\n", current);

    forwardPage();
    printf("After Forward: %s\n", current);

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Current: Wikipedia
After Back: GitHub
After Forward: Wikipedia
```

4. Sort a Stack Using a Temporary Stack

```
#include <stdio.h>

#define MAX 50

int s[MAX], t[MAX];
int top = -1, tempTop = -1;

void push(int a[], int *top, int x) {
    a[++(*top)] = x;
}

int pop(int a[], int *top) {
    return a[(*top)--];
}

int main() {
    int n, i, x;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        push(s, &top, x);
    }

    while (top >= 0) {
        x = pop(s, &top);

        while (tempTop >= 0 && t[tempTop] > x)
            push(s, &top, pop(t, &tempTop));

        push(t, &tempTop, x);
    }

    printf("Sorted stack: ");
    while (tempTop >= 0)
        printf("%d ", pop(t, &tempTop));

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Enter number of elements: 6
Enter elements: 31 7 24 15 42 10
Sorted stack: 7 10 15 24 31 42
```

5. Stock Span Problem

```
#include <stdio.h>

#define MAX 100

int main() {
    int price[MAX], span[MAX], st[MAX];
    int n, top = -1, i;

    printf("Enter number of days: ");
    scanf("%d", &n);

    printf("Enter prices: ");
    for (i = 0; i < n; i++)
        scanf("%d", &price[i]);

    for (i = 0; i < n; i++) {
        while (top >= 0 && price[st[top]] <= price[i])
            top--;

        span[i] = (top < 0) ? i + 1 : i - st[top];
        st[++top] = i;
    }

    printf("Span: ");
    for (i = 0; i < n; i++)
        printf("%d ", span[i]);

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Enter number of days: 7
Enter prices: 90 75 60 65 80 78 95
Span: 1 1 1 2 4 1 7
```

6. Nearest Element to Right with Greater Frequency

```c
#include <stdio.h>

#define MAX 100

int main() {
    int a[MAX], freq[MAX] = {0}, ans[MAX], st[MAX];
    int n, i, j, top = -1;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        if (a[i] >= 0 && a[i] < MAX)
            freq[a[i]]++;
    }

    for (i = n - 1; i >= 0; i--) {
        while (top >= 0 &&
               freq[a[st[top]]] <= freq[a[i]])
            top--;

        ans[i] = (top < 0) ? -1 : a[st[top]];
        st[++top] = i;
    }

    printf("Output: ");
    for (i = 0; i < n; i++)
        printf("%d ", ans[i]);

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Enter number of elements: 6
Enter elements: 4 5 5 4 4 6
Output: -1 4 4 -1 -1 -1
```

7. I/D Pattern Using Stack

```
#include <stdio.h>
#include <string.h>

#define MAX 50

int main() {
    char p[MAX];
    int st[MAX], top = -1, num = 1, i;

    printf("Enter pattern (I/D): ");
    scanf("%s", p);

    for (i = 0; i <= (int)strlen(p); i++) {
        st[++top] = num++;

        if (p[i] == 'I' || p[i] == '\0') {
            while (top >= 0)
                printf("%d", st[top--]);
        }
    }

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Enter pattern (I/D): DIDDI
214365
```

8. Minimum Possible Sum of Two Numbers

```c
#include <stdio.h>

void sort(int a[], int n) {
    int i, j, t;
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (a[i] > a[j]) {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
}

int main() {
    int a[20], n, i;
    long n1 = 0, n2 = 0;

    printf("Number of digits: ");
    scanf("%d", &n);

    printf("Enter digits: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    sort(a, n);

    for (i = 0; i < n; i++) {
        if (i % 2 == 0)
            n1 = n1 * 10 + a[i];
        else
            n2 = n2 * 10 + a[i];
    }

    printf("Numbers: %ld and %ld\n", n1, n2);
    printf("Minimum sum = %ld\n", n1 + n2);

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Number of digits: 6
Enter digits: 9 1 4 6 2 8
Numbers: 246 and 189
Minimum sum = 435
```

9. First Negative Integer in Every Window of Size K

```c
#include <stdio.h>

#define MAX 100

int main() {
    int a[MAX], q[MAX];
    int n, k, front = 0, rear = 0, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Window size: ");
    scanf("%d", &k);

    for (i = 0; i < n; i++) {
        if (a[i] < 0)
            q[rear++] = i;

        if (i >= k - 1) {
            while (front < rear && q[front] <= i - k)
                front++;

            if (front < rear)
                printf("%d ", a[q[front]]);
            else
                printf("0 ");
        }
    }

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Enter number of elements: 7
Enter elements: 12 -5 7 -9 4 8 -3
Window size: 3
-5 -5 -9 -9 -3
```

10. Rotten Oranges Using Queue

```c
#include <stdio.h>

#define MAX 50

struct Cell {
    int r, c;
};

int main() {
    int a[MAX][MAX], n, m, fresh = 0;
    struct Cell q[MAX * MAX];
    int front = 0, rear = 0, minutes = 0;
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};
    int i, j;

    printf("Rows and columns: ");
    scanf("%d %d", &n, &m);

    printf("Enter matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) {
            scanf("%d", &a[i][j]);

            if (a[i][j] == 2)
                q[rear++] = (struct Cell){i, j};
            else if (a[i][j] == 1)
                fresh++;
        }

    while (front < rear && fresh > 0) {
        int count = rear - front;

        while (count--) {
            int r = q[front].r, c = q[front].c;
            front++;

            for (j = 0; j < 4; j++) {
                int nr = r + dr[j], nc = c + dc[j];

                if (nr >= 0 && nr < n && nc >= 0 && nc < m &&
                    a[nr][nc] == 1) {
                    a[nr][nc] = 2;
                    fresh--;
                    q[rear++] = (struct Cell){nr, nc};
                }
            }
        }
        minutes++;
    }

    if (fresh == 0)
        printf("Minimum time = %d\n", minutes);
    else
        printf("Not all oranges can become rotten.\n");

    return 0;
}
```

Sample Output

```
Homework Set 4 — Program Output
```

```
Rows and columns: 3 4
Enter matrix:
2 1 0 2
1 1 1 0
0 1 1 1
Minimum time = 4
```